



UNIVERSIDADE FEDERAL DO CEARÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA ELÉTRICA
CURSO DE GRADUAÇÃO EM ENGENHARIA ELÉTRICA

GABRIEL RUFINO MONTENEGRO

ANÁLISE COMPARATIVA DE FORMULAÇÕES DE FLUXO DE POTÊNCIA ÓTIMO
COM AÇÕES DE CONTROLE EM JUMP/JULIA

FORTALEZA

2026

GABRIEL RUFINO MONTENEGRO

ANÁLISE COMPARATIVA DE FORMULAÇÕES DE FLUXO DE POTÊNCIA ÓTIMO COM
AÇÕES DE CONTROLE EM JUMP/JULIA

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Engenharia Elétrica do
Centro de Tecnologia da Universidade Federal
do Ceará, como requisito parcial à obtenção do
grau de bacharel em Engenharia Elétrica.

Orientador: Prof. Dr. Lucas Silveira
Melo

FORTALEZA

2026

GABRIEL RUFINO MONTENEGRO

ANÁLISE COMPARATIVA DE FORMULAÇÕES DE FLUXO DE POTÊNCIA ÓTIMO COM
AÇÕES DE CONTROLE EM JUMP/JULIA

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Engenharia Elétrica do
Centro de Tecnologia da Universidade Federal
do Ceará, como requisito parcial à obtenção do
grau de bacharel em Engenharia Elétrica.

Aprovada em:

BANCA EXAMINADORA

Prof. Dr. Lucas Silveira Melo (Orientador)
Universidade Federal do Ceará (UFC)

Aos meus pais e ao meu irmão Davi.

AGRADECIMENTOS

Aos meus pais, José Fábio Bezerra Montenegro e Célia Maria Rufino Montenegro, por tudo que me proporcionaram ao longo da vida — pelo amor incondicional, pelos valores que me ensinaram e por nunca medirem esforços para que eu tivesse as condições necessárias para crescer e realizar meus objetivos. Tudo que conquisto é reflexo do que vocês construíram para mim.

Ao meu irmão mais velho, Davi, por ser uma referência constante em minha vida. Sua trajetória sempre me inspirou a dar o melhor de mim e a não desistir diante das dificuldades.

Aos meus amigos da UFC, Gabriel Fontenele, Kevyn Wenzel, Livia Belo e Yann Bruno, pela companhia, pelo apoio e por tornarem essa jornada muito mais leve e significativa. A graduação não teria sido a mesma sem a presença de vocês.

Aos amigos que fiz durante meu período de duplo diploma na França, em especial Lucas, Oliver, Rezende, Renato e Lara, que foram meu ponto de apoio em um momento tão intenso quanto viver e estudar em outro país. A amizade de vocês foi essencial para superar as dificuldades daquela experiência e torná-la uma das mais marcantes da minha vida.

Ao meu orientador, Prof. Dr. Lucas Silveira Melo, pelo acompanhamento dedicado, pela orientação precisa e pela confiança depositada em mim durante o desenvolvimento deste trabalho. Sua contribuição foi fundamental para a qualidade desta pesquisa.

Ao PET Engenharia Elétrica da UFC, por todo o desenvolvimento acadêmico, profissional e pessoal que me proporcionou. As experiências vividas no grupo foram determinantes na minha formação como engenheiro.

Ao Doutorando em Engenharia Elétrica, Ednardo Moreira Rodrigues, e seu assistente, Alan Batista de Oliveira, aluno de graduação em Engenharia Elétrica, pela adequação do *template* utilizado neste trabalho para que o mesmo ficasse de acordo com as normas da biblioteca da Universidade Federal do Ceará (UFC).

“O sonho é que leva a gente para frente. Se a gente for seguir a razão, fica aquietado, acomodado.”

(Ariano Suassuna)

RESUMO

O Fluxo de Potência Ótimo (FPO) é uma das ferramentas centrais para o planejamento e a operação de Sistemas Elétricos de Potência (SEP). No contexto brasileiro, programas consolidados de mercado, como o ANAREDE do CEPEL, incorporam ações de controle específicas — limites de potência reativa de geradores (QLIM), limites de magnitude de tensão em barras (VLIM), controle automático de susceptância de bancos chaveados (CSCA), controle automático de *tap* de transformadores (CTAP) e agregadores de geração distribuída (DERA) — que não estão presentes de forma nativa nas formulações acadêmicas clássicas implementadas em pacotes *open-source*. Este trabalho apresenta a construção, em linguagem Julia, de uma progressão incremental de seis formulações de fluxo de potência AC: duas formulações de referência baseadas no pacote *PowerModels.jl* (uma de FPO completo e uma de fluxo de potência clássico) e quatro formulações *hand-built* em *JuMP*, nas quais cada uma das ações de controle ANAREDE listadas é introduzida sequencialmente sobre a anterior. As entradas são lidas a partir de arquivos no formato PWF utilizando o pacote *PWF.jl*, com extração das estruturas de controle (DOPC e DLIN) habilitada. O modelo não linear é resolvido com o *solver* Ipopt e os controles são introduzidos via variáveis de folga penalizadas em uma função objetivo do tipo *soft-constraint*. As seis formulações são submetidas a uma bateria padronizada de vinte e cinco casos de teste, abrangendo redes pedagógicas de três a nove barras, redes de médio porte (300 e 500 barras) e um *snapshot* do Sistema Interligado Nacional (caso *01 MAXIMA NOTURNA_DEZ25*, com cerca de seis mil barras). Os resultados mostram que as formulações *hand-built* convergem em uma faixa estritamente maior de casos do que as referências *PowerModels* —em particular, recuperam soluções factíveis em casos com QLIM ativo nos quais o FPO clássico declara o problema infactível —e que as soluções produzidas pelas formulações com CSCA e CTAP se agrupam em *clusters* numericamente distintos da referência, reproduzindo o comportamento esperado das heurísticas de controle do ANAREDE.

Palavras-chave: Fluxo de potência ótimo. Sistemas elétricos de potência. JuMP. PowerModels.jl. Ações de controle. Ipopt.

ABSTRACT

The Optimal Power Flow (OPF) problem is a central tool for the planning and operation of Electric Power Systems (EPS). In the Brazilian context, established industry-grade tools such as CEPEL's ANAREDE incorporate specific control actions —generator reactive power limits (QLIM), bus voltage magnitude limits (VLIM), automatic switched-shunt susceptance control (CSCA), automatic on-load transformer tap control (CTAP) and distributed energy resource aggregators (DERA) —that are not natively available in the classical academic formulations implemented by mainstream open-source packages. This work presents the construction, in the Julia language, of an incremental progression of six AC power flow formulations: two reference formulations based on the *PowerModels.jl* package (a full OPF and a classical power flow) and four hand-built *JuMP* formulations in which each of the ANAREDE control actions is sequentially introduced on top of the previous one. Inputs are read from PWF files using the *PWF.jl* package, with extraction of the control structures (DOPC and DLIN) enabled. The non-linear model is solved with the Ipopt solver and the controls are introduced via penalized slack variables in a soft-constraint objective. The six formulations are submitted to a standardized test battery of twenty-five cases, comprising pedagogical networks of three to nine buses, medium-scale networks (300 and 500 buses) and a snapshot of the Brazilian Interconnected Power System (the *01 MAXIMA NOTURNA_DEZ25* case, with approximately six thousand buses). Results show that the hand-built formulations converge on a strictly larger set of cases than the *PowerModels* references —in particular, they recover feasible solutions on QLIM-active cases where the classical OPF declares the problem infeasible —and that the solutions produced by the CSCA- and CTAP-augmented formulations cluster in numerically distinct groups from the reference, reproducing the expected behavior of ANAREDE's control heuristics.

Keywords: Optimal power flow. Electric power systems. JuMP. PowerModels.jl. Control actions. Ipopt.

LISTA DE FIGURAS

Figura 1 – Modelo π de um ramo com transformador	17
Figura 2 – Fluxograma do pipeline de execução das formulações	24
Figura 3 – Mapa de calor da convergência das formulações (14)–(19) na bateria de testes	31
Figura 4 – Topologia da rede-exemplo 3bus_DCSC	31
Figura 5 – Perfil de tensão por barra no caso 3bus_DCSC	32

LISTA DE TABELAS

Tabela 1 – Progressão das formulações implementadas	24
Tabela 2 – Convergência global e <i>clusters</i> de equivalência das formulações (14)–(19) na bateria	30
Tabela 3 – Divergência barra a barra no caso 3bus_DCSC	32

LISTA DE SÍMBOLOS

$\mathcal{N}$	Conjunto de barras (nós) da rede
$\mathcal{G}$	Conjunto de geradores
$\mathcal{L}$	Conjunto de cargas
$\mathcal{B}$	Conjunto de ramos AC (linhas e transformadores)
$\mathcal{B}_{dc}$	Conjunto de elos HVDC
$\mathcal{S}$	Conjunto de bancos <i>shunt</i> chaveados
V_i	Magnitude de tensão na barra i , em p.u.
θ_i	Ângulo de tensão na barra i , em radianos
p_k^g, q_k^g	Potências ativa e reativa geradas pelo gerador k , em p.u.
p_l^d, q_l^d	Potências ativa e reativa demandadas pela carga l , em p.u.
g_{ij}, b_{ij}	Condutância e susceptância série do ramo (i, j) , em p.u.
b_s^{sh}	Susceptância <i>shunt</i> ajustável do banco s , em p.u.
t_{ij}	Razão de transformação (módulo do <i>tap</i>) do transformador no ramo (i, j)
s_i^v	Variável de folga associada ao controle VLIM na barra i , em p.u.
s_l^q	Variável de folga associada ao controle QLIM na carga l , em p.u.
ρ	Penalidade associada às variáveis de folga na função objetivo
$V_i^{\min}, V_i^{\max}$	Limites inferior e superior de magnitude de tensão na barra i
$q_k^{g, \min}, q_k^{g, \max}$	Limites inferior e superior de geração reativa do gerador k

SUMÁRIO

1	INTRODUÇÃO	14
1.1	Motivação	14
1.2	Objetivos	15
1.2.1	<i>Objetivo geral</i>	15
1.2.2	<i>Objetivos específicos</i>	15
1.3	Organização do trabalho	16
2	FUNDAMENTAÇÃO TEÓRICA	17
2.1	Modelagem da rede elétrica	17
2.2	Fluxo de Potência AC	18
2.3	Fluxo de Potência Ótimo	18
2.3.1	<i>Relaxações do FPO</i>	19
2.4	Ações de controle do programa ANAREDE	20
2.4.1	<i>QLIM —limites de geração reativa</i>	20
2.4.2	<i>VLIM —limites de magnitude de tensão</i>	20
2.4.3	<i>CSCA —bancos shunt chaveados</i>	21
2.4.4	<i>CTAP —tap ajustável de transformadores</i>	21
2.4.5	<i>DERA —agregação de geração distribuída</i>	21
2.4.6	<i>Síntese das ações de controle</i>	21
3	METODOLOGIA	22
3.1	Ambiente computacional	22
3.2	Pipeline de execução	23
3.3	Formulações implementadas	24
3.3.1	<i>Formulação (14): FPO AC de referência</i>	25
3.3.2	<i>Formulação (15): FP AC clássico</i>	25
3.3.3	<i>Formulação (16): FP controlado QLIM + VLIM</i>	25
3.3.4	<i>Formulação (17): adiciona CSCA</i>	26
3.3.5	<i>Formulação (18): adiciona CTAP</i>	26
3.3.6	<i>Formulação (19): adiciona DERA</i>	27
3.4	Bateria de casos e critérios de comparação	27
3.4.1	<i>Casos de teste</i>	27

3.4.2	<i>Cr�terios de compara��o</i>	28
4	RESULTADOS	29
4.1	Converg�ncia global e clusters	29
4.2	An�lise barra a barra	30
4.3	Discuss�o	33
4.3.1	<i>Comportamento t�pico das a��es de controle</i>	33
4.3.2	<i>O caso SIN</i>	33
4.3.3	<i>Casos sint�ticos com erro de modelo</i>	34
5	CONCLUS�ES E TRABALHOS FUTUROS	35
5.1	S�ntese das contribui��es	35
5.2	Limita��es	36
5.3	Trabalhos futuros	36
	REFER�NCIAS	38
	AP�NDICES	39
	AP�NDICE A – Trecho da implementa��o JuMP da formula��o (16) . . .	39

1 INTRODUÇÃO

A operação segura, confiável e econômica de Sistemas Elétricos de Potência (SEP) é um dos maiores desafios da engenharia moderna. Com o crescimento contínuo da demanda por energia, a inserção massiva de fontes renováveis intermitentes e a dimensão continental do Sistema Interligado Nacional (SIN), a análise de redes elétricas tornou-se um problema de larga escala cuja resolução depende, simultaneamente, de modelos matemáticos rigorosos e de implementações computacionais eficientes. Nesse cenário, o estudo de Fluxo de Potência (FP) e a sua extensão como problema de otimização —Fluxo de Potência Ótimo (FPO) —são as duas ferramentas centrais sobre as quais o planejamento e a operação dos SEP são executados (MEIER, 2006; GLOVER *et al.*, 2012).

O FP em corrente alternada (CA) é classicamente formulado como um sistema de equações algébricas não lineares e não convexas, derivado da aplicação das Leis de Kirchhoff na rede elétrica. O FPO acrescenta a esse sistema uma função objetivo, tipicamente o custo de geração ou as perdas ativas totais, e um conjunto de restrições de desigualdade que codificam os limites operacionais dos equipamentos. Em sua forma natural, o FPO pertence à classe de problemas de programação não linear (NLP) não convexas, cuja resolução em redes de grande porte exige métodos numéricos avançados, formulações cuidadosas e *solvers* robustos (MOLZAHN; HISKENS, 2019).

1.1 Motivação

A análise de SEP no contexto brasileiro impõe particularidades que vão além do FPO acadêmico. A operação do SIN é fortemente dependente de **ações de controle** —limites de geração reativa em barras PV (QLIM), limites de magnitude de tensão (VLIM), bancos *shunt* chaveados (CSCA), *taps* ajustáveis de transformadores em carga (CTAP) e, mais recentemente, agregadores de geração distribuída (DERA) —que precisam ser representadas de forma explícita para que os pontos de operação calculados em ambiente computacional reproduzam o comportamento real da rede. Programas consolidados na indústria nacional, com destaque para o ANAREDE do Centro de Pesquisas de Energia Elétrica (CEPEL) (CENTRO DE PESQUISAS DE ENERGIA ELÉTRICA, 2023), incorporam esses controles por meio de heurísticas e laços iterativos específicos, codificados em uma cultura de modelagem que está documentada principalmente nos manuais e em arquivos de dados no formato PWF.

Por outro lado, o ecossistema acadêmico de pesquisa em otimização de SEP tem se consolidado em torno de pacotes *open-source* de propósito geral, dentre os quais o *PowerModels.jl*, escrito na linguagem Julia, é hoje referência na pesquisa internacional. O *PowerModels.jl* oferece formulações limpas para o FPO AC e para suas relaxações DC, SOC e SDP, mas não possui suporte nativo às ações de controle ANAREDE descritas no parágrafo anterior. Assim, surge uma lacuna metodológica relevante: *como reproduzir, em uma ferramenta moderna e open-source como o JuMP, o conjunto de ações de controle que tornam o FPO compatível com a prática de operação do SIN?*

Essa lacuna tem sido tratada, no grupo de pesquisa do orientador, pelo desenvolvimento do pacote *ControlPowerFlow.jl*, que propõe uma implementação modular dessas ações de controle em Julia. O presente trabalho situa-se como uma contribuição complementar a esse esforço: em vez de utilizar a biblioteca como caixa-preta, são construídas *do zero*, em *JuMP*, formulações de FP e FPO que incorporam, uma a uma, as mesmas ações de controle —de modo que cada formulação possa ser comparada, ponto a ponto, contra as duas referências disponíveis no *PowerModels.jl* (FPO clássico e FP clássico) sobre a mesma bateria de casos.

1.2 Objetivos

1.2.1 Objetivo geral

Construir, validar e comparar uma progressão incremental de formulações de fluxo de potência AC, escritas em *JuMP/Julia* sobre o *solver* Ipopt, que incorporam de forma explícita as principais ações de controle utilizadas pelo programa ANAREDE —QLIM, VLIM, CSCA, CTAP e DERA —sobre uma bateria de casos que vai de redes pedagógicas de três barras até um *snapshot* do SIN.

1.2.2 Objetivos específicos

- compreender e documentar a formulação matemática do FP e do FPO em coordenadas polares, bem como as principais relaxações disponíveis na literatura;
- utilizar a biblioteca *PWF.jl* para a leitura de arquivos no formato ANAREDE, com extração das estruturas de controle (registros DOPC e DLIN);
- implementar duas formulações de referência baseadas em *PowerModels.jl* —uma de FPO completo (*solve_ac_opf*) e uma de FP clássico (*solve_ac_pf*) —para

servirem como *baseline* de comparação;

- implementar quatro formulações *hand-built* em JuMP que adicionam, sequencialmente, as ações de controle QLIM+VLIM, CSCA, CTAP e DERA, sempre via variáveis de folga penalizadas em uma função objetivo do tipo *soft-constraint*;
- submeter as seis formulações a uma bateria padronizada de vinte e cinco casos de teste, abrangendo redes pedagógicas, redes de médio porte e o SIN, e analisar o comportamento de convergência, o agrupamento das soluções em *clusters* numericamente equivalentes e os desvios barra a barra em relação às referências.

1.3 Organização do trabalho

Para uma melhor compreensão e encadeamento lógico, este documento está estruturado em cinco capítulos. O Capítulo 2 apresenta a fundamentação teórica do FP e do FPO em coordenadas polares, discute as principais relaxações da literatura e revisa as ações de controle que motivam o trabalho. O Capítulo 3 detalha a metodologia adotada: a arquitetura de software em Julia, o uso do *PWF.jl* para leitura de dados, a configuração do *solver* Ipopt e, principalmente, a formulação matemática das seis variantes de FP/FPO implementadas. No Capítulo 4 são expostos os resultados da bateria de testes —a tabela global de convergência, o agrupamento das soluções em *clusters* numericamente equivalentes e a análise de desvios barra a barra. Por fim, o Capítulo 5 compila as conclusões alcançadas e propõe direcionamentos para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

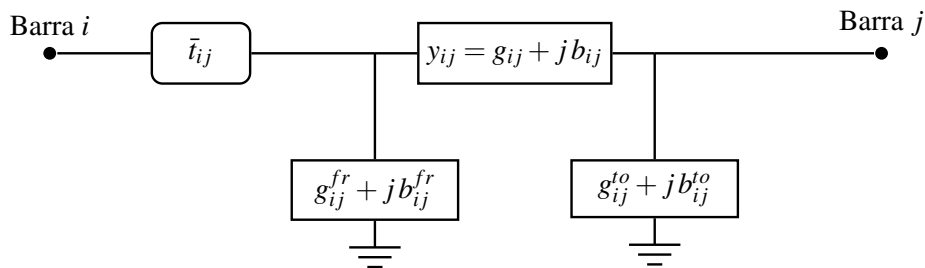
Este capítulo apresenta a base conceitual sobre a qual todo o restante do trabalho está construído. A seção 2.1 define o modelo π de rede e fixa a notação utilizada. A seção 2.2 introduz o problema de Fluxo de Potência (FP) AC em coordenadas polares. A seção 2.3 estende o FP ao Fluxo de Potência Ótimo (FPO) e discute brevemente as principais relaxações disponíveis na literatura. Por fim, a seção 2.4 revisa as ações de controle do programa ANAREDE que motivam a construção das formulações *hand-built* apresentadas no Capítulo 3.

2.1 Modelagem da rede elétrica

A rede elétrica é representada por um grafo $\mathcal{G} = (\mathcal{N}, \mathcal{B})$, em que o conjunto de nós $\mathcal{N}$ corresponde às barras do sistema e o conjunto de arestas $\mathcal{B}$ aos ramos AC, abrangendo tanto linhas de transmissão quanto transformadores em fase. Adicionalmente, $\mathcal{B}_{dc}$ representa elos HVDC, $\mathcal{G}$ o conjunto de geradores, $\mathcal{L}$ o conjunto de cargas e $\mathcal{S}$ o conjunto de bancos *shunt*, cada elemento associado a uma barra de $\mathcal{N}$.

Cada ramo $(i, j) \in \mathcal{B}$ é descrito pelo modelo π clássico (Figura 1), com admitância série $y_{ij} = g_{ij} + jb_{ij}$, susceptâncias *shunt* de extremidade b_{ij}^{fr} e b_{ij}^{to} , condutâncias *shunt* de extremidade g_{ij}^{fr} e g_{ij}^{to} e uma razão de transformação complexa $\bar{t}_{ij} = t_{ij} e^{j\phi_{ij}}$ que generaliza linhas e transformadores. Para uma linha simples, $t_{ij} = 1$ e $\phi_{ij} = 0$.

Figura 1 – Modelo π de um ramo com transformador



Fonte: elaborado pelo autor.

A tensão complexa em cada barra é representada na forma fasorial $\bar{V}_i = V_i e^{j\theta_i}$, onde V_i é a magnitude (em p.u.) e θ_i é o ângulo (em radianos). Esta é a representação polar adotada em todas as formulações deste trabalho. A representação complementar em coordenadas retangulares, $\bar{V}_i = e_i + jf_i$, embora útil em algumas relaxações, não será utilizada na implementação.

2.2 Fluxo de Potência AC

O problema de FP consiste em determinar o estado da rede $(V_i, \theta_i)_{i \in \mathcal{N}}$ a partir de injeções e tensões especificadas em cada barra. Classicamente, as barras são tipadas em três categorias: *slack* (referência de ângulo, com V e θ fixados), PV (com V e p^g especificados) e PQ (com p^g e q^g especificados, sendo V e θ as incógnitas) (MEIER, 2006; GLOVER *et al.*, 2012).

Aplicando-se a Lei das Correntes de Kirchhoff em cada barra $i \in \mathcal{N}$ e expressando-se as injeções em termos da matriz de admitâncias nodais $\bar{Y}$, obtêm-se as duas equações de balanço de potência *ativa* e *reativa*:

$$\sum_{(i,j) \in \mathcal{B}_i} p_{ij} = \sum_{k \in \mathcal{G}_i} p_k^g - \sum_{l \in \mathcal{L}_i} p_l^d - g_i^{sh} V_i^2, \quad (2.1)$$

$$\sum_{(i,j) \in \mathcal{B}_i} q_{ij} = \sum_{k \in \mathcal{G}_i} q_k^g - \sum_{l \in \mathcal{L}_i} q_l^d + b_i^{sh} V_i^2, \quad (2.2)$$

onde $\mathcal{B}_i$ denota o conjunto de arcos incidentes à barra i , $\mathcal{G}_i$ e $\mathcal{L}_i$ os geradores e cargas conectados a i , e g_i^{sh} , b_i^{sh} as condutância e susceptância *shunt* agregadas na barra. Os fluxos p_{ij} e q_{ij} no terminal de envio do ramo (i, j) são dados, no modelo π com transformador, por:

$$p_{ij} = \frac{g_{ij} + g_{ij}^{fr}}{t_{ij}^2} V_i^2 - \frac{1}{t_{ij}} \left[g_{ij} \cos(\theta_i - \theta_j - \phi_{ij}) + b_{ij} \sin(\theta_i - \theta_j - \phi_{ij}) \right] V_i V_j, \quad (2.3)$$

$$q_{ij} = -\frac{b_{ij} + b_{ij}^{fr}}{t_{ij}^2} V_i^2 - \frac{1}{t_{ij}} \left[g_{ij} \sin(\theta_i - \theta_j - \phi_{ij}) - b_{ij} \cos(\theta_i - \theta_j - \phi_{ij}) \right] V_i V_j. \quad (2.4)$$

Expressões análogas valem para o terminal (j, i) , com a respectiva troca de subscritos e a substituição de g^{fr}, b^{fr} por g^{to}, b^{to} . As Equações 2.1–2.4 formam um sistema algébrico não linear em (V_i, θ_i) , tradicionalmente resolvido por métodos iterativos do tipo Newton-Raphson ou Gauss-Seidel.

2.3 Fluxo de Potência Ótimo

O FPO estende o FP transformando $(V_i, \theta_i, p_k^g, q_k^g)$ em variáveis de decisão, sujeitas às equações de balanço (2.1)–(2.2) como restrições de igualdade, e adicionando restrições de

desigualdade que codificam os limites operacionais dos equipamentos. Em sua forma canônica AC (MOLZAHN; HISKENS, 2019), o problema escreve-se como:

$$\begin{aligned}
 & \min_{V, \theta, p^g, q^g} \sum_{k \in \mathcal{G}} c_k(p_k^g) \\
 & \text{s.a. Equações de balanço (2.1), (2.2),} \\
 & V_i^{\min} \leq V_i \leq V_i^{\max}, \quad \forall i \in \mathcal{N}, \\
 & p_k^{g, \min} \leq p_k^g \leq p_k^{g, \max}, \quad \forall k \in \mathcal{G}, \\
 & q_k^{g, \min} \leq q_k^g \leq q_k^{g, \max}, \quad \forall k \in \mathcal{G}, \\
 & |s_{ij}| \leq s_{ij}^{\max}, \quad \forall (i, j) \in \mathcal{B}, \\
 & \theta_{\text{slack}} = 0,
 \end{aligned} \tag{2.5}$$

onde $c_k(\cdot)$ é a curva de custo do gerador k —em geral quadrática, $c_k(p_k^g) = a_k(p_k^g)^2 + b_k p_k^g + c_k^0$ —e $s_{ij}^{\max}$ é o limite de aparente do ramo. O ponto a destacar é que o FPO AC, devido à presença das equações (2.3)–(2.4), é um problema de programação não linear (NLP) *não convexo*, o que abre espaço para múltiplos ótimos locais e exige cuidados na escolha do *solver* e das condições iniciais.

2.3.1 Relaxações do FPO

A não convexidade do FPO motivou, ao longo das últimas duas décadas, uma extensa literatura sobre relaxações e aproximações que tornam o problema tratável, com destaque para três famílias (MOLZAHN; HISKENS, 2019):

- **Aproximação DC** —linearização das equações de fluxo sob as hipóteses de tensões próximas a 1 p.u., ângulos pequenos e perdas desprezíveis. Resulta em um problema de programação linear (LP) de baixíssimo custo computacional, amplamente utilizado em planejamento de longo prazo;
- **Relaxação SOC** —reformulação do FPO em termos de variáveis $W_{ij} = V_i V_j \cos(\theta_i - \theta_j)$ e $T_{ij} = V_i V_j \sin(\theta_i - \theta_j)$, na qual as restrições não convexas são substituídas por desigualdades de cone de segunda ordem. É exata em redes radiais e fornece bons *lower bounds* em redes malhadas;
- **Relaxação SDP** —reformulação como problema de programação semidefinida sobre a matriz $W = VV^H$, mais apertada que a SOC mas computacionalmente mais cara.

Embora o presente trabalho não utilize as relaxações DC, SOC ou SDP em sua progressão principal de formulações —que mantém a formulação AC polar não convexa —uma comparação isolada entre as três famílias é apresentada em script auxiliar (*CodeComparing.jl*) sobre o caso `case5.m`, com o objetivo didático de evidenciar o *gap* entre a relaxação e a solução AC.

2.4 Ações de controle do programa ANAREDE

As formulações clássicas de FP e FPO descritas nas seções anteriores tratam V_i , θ_i , p_k^g e q_k^g como variáveis livres dentro de seus limites caixa. A operação real de um SEP, no entanto, envolve um conjunto de **ações de controle** que se acoplam a essas variáveis e que, no programa ANAREDE (CHÁVARRY, 2024) —referência da indústria brasileira —são tratadas por meio de heurísticas e laços iterativos específicos. Este trabalho considera cinco dessas ações, nomeadas a seguir conforme a sigla utilizada nos arquivos PWF.

2.4.1 QLIM —limites de geração reativa

Em uma barra PV, a tensão V_i é mantida em um *setpoint* V_i^{ref} enquanto o gerador associado é capaz de prover a potência reativa necessária. Quando q_k^g atinge $q_k^{g,max}$ (ou $q_k^{g,min}$), a barra é convertida em PQ, q_k^g é fixado no limite e V_i passa a flutuar. Esse comportamento de *type-switching* é central para a operação realista, mas é *descontínuo* e, quando codificado diretamente, conflita com solvers NLP suaves como o Ipopt. Uma forma comum de contorná-lo, adotada neste trabalho, é introduzir uma variável de folga s_l^q no balanço reativo da carga e penalizá-la na função objetivo, mantendo q_k^g dentro dos limites caixa originais.

2.4.2 VLIM —limites de magnitude de tensão

De forma análoga, o controle VLIM trata V_i como variável dentro dos limites caixa $[V_i^{min}, V_i^{max}]$, mas adiciona uma restrição de igualdade entre V_i e o *setpoint* V_i^{ref} por meio de uma variável de folga s_i^v , também penalizada. Combinado com QLIM, o par de slacks (s_i^v, s_l^q) implementa, na linguagem do otimizador, a mesma lógica que o ANAREDE expressa no laço de *type-switching*.

2.4.3 CSCA —bancos shunt chaveados

Os bancos *shunt* (capacitivos ou indutivos) chaveáveis são representados, no PWF, por uma susceptância b_s^{sh} que pode assumir um conjunto discreto de valores entre $b_s^{sh,\min}$ e $b_s^{sh,\max}$. No ANAREDE, o controle CSCA promove o chaveamento dos passos para manter V_i próxima do *setpoint*. Em uma formulação NLP, b_s^{sh} é tipicamente relaxada para variável contínua dentro do intervalo $[b_s^{sh,\min}, b_s^{sh,\max}]$ —representação que será mantida neste trabalho —e a discretização posterior é tratada via *rounding* ou em formulação MINLP.

2.4.4 CTAP —tap ajustável de transformadores

Transformadores em carga (*on-load tap-changer*, OLTC) possuem razão de transformação t_{ij} ajustável, em geral para regulação de tensão na barra do lado de baixa. No PWF, t_{ij} é dada com um valor inicial e limites $[t_{ij}^{\min}, t_{ij}^{\max}]$. O controle CTAP é, novamente, discreto na operação real (passos de 0,5% a 1%), mas a relaxação contínua é a representação adotada na formulação NLP utilizada neste trabalho, com uma restrição adicional ligando V_i ao *setpoint* via folga.

2.4.5 DERA —agregação de geração distribuída

DERA (*Distributed Energy Resource Aggregator*) é a representação ANAREDE para conjuntos de geração distribuída —tipicamente fotovoltaica ou eólica em distribuição —agregados em uma barra de transmissão. Cada DERA aporta uma injeção (p_d^{der}, q_d^{der}) que pode ser fixada ou tratada como variável adicional. Neste trabalho, a representação adotada é minimalista: o DERA acrescenta termos de injeção ao balanço de potência da barra hospedeira, sem decisões de despacho próprias.

2.4.6 Síntese das ações de controle

As cinco ações descritas nas Seções 2.4.1–2.4.5 compõem o conjunto de controles que serão sucessivamente adicionados, no Capítulo 3, sobre uma formulação base de FP em JuMP. A cada controle adicionado, a função objetivo *soft-constraint* ganha um novo termo de penalidade, e o conjunto de variáveis de decisão cresce em $|\mathcal{S}|$ (CSCA), $|\mathcal{B}_{tap}|$ (CTAP) ou $|\mathcal{D}|$ (DERA), conforme o caso. Esta organização incremental permite isolar, no Capítulo 4, o impacto de cada controle sobre a convergência e sobre o ponto de operação calculado.

3 METODOLOGIA

Este capítulo descreve o pipeline de software construído para implementar e comparar as seis formulações de fluxo de potência consideradas neste trabalho. A seção 3.1 apresenta o ambiente computacional em Julia e os pacotes utilizados. A seção 3.2 descreve o pipeline de execução, desde a leitura do arquivo PWF até a exportação dos resultados. A seção 3.3 é o núcleo do capítulo: nela, é detalhada a formulação matemática de cada uma das seis variantes —duas baseadas em *PowerModels.jl* e quatro *hand-built* em JuMP —que, juntas, formam a progressão (14)→(19) usada no Capítulo 4. Por fim, a seção 3.4 descreve a bateria de vinte e cinco casos de teste e os critérios de comparação adotados.

3.1 Ambiente computacional

Toda a implementação foi realizada na linguagem Julia (BEZANSON *et al.*, 2017) (versão 1.x, série estável), escolhida por aliar uma sintaxe de alto nível, voltada para computação científica, a um desempenho computacional comparável ao de linguagens compiladas. O ambiente de pacotes utilizado, definido em `Project.toml` e `Manifest.toml` na raiz do repositório, agrega:

- *PWF.jl* —leitura e parsing de arquivos no formato ANAREDE PWF, com extração de estruturas de controle (registros DOPC e DLIN) habilitada via `add_control_data = true`;
- *PowerModels.jl* (COFFRIN *et al.*, 2018) —construção da estrutura de referência (`build_ref`) sobre os dados parseados e oferta das funções `solve_ac_opf` e `solve_ac_pf` que implementam as duas formulações de referência;
- *JuMP.jl* (LUBIN *et al.*, 2023) —linguagem de modelagem matemática usada para escrever, à mão, as quatro formulações controladas;
- *Ipopt.jl* (WÄCHTER; BIEGLER, 2006) —interface Julia para o *solver* de programação não linear baseado em método de pontos interiores;
- *CSV.jl* e *DataFrames.jl* —exportação dos resultados para arquivos CSV.

A escolha do Ipopt como *solver* se justifica por três fatores: (i) é *open-source* e amplamente utilizado em problemas NLP de SEP; (ii) suporta nativamente expressões não lineares no JuMP via `@NLexpression` e `@NLconstraint`; e (iii) é o *solver* adotado por padrão pelo *PowerModels.jl* em suas formulações AC, garantindo uma comparação justa entre as refe-

rências e as formulações *hand-built*. As opções utilizadas são `max_iter = 3000`, `tol = 1e-5` e `print_level = 5`. Esses valores foram escolhidos de forma permissiva por causa do porte do caso *01 MAXIMA NOTURNA_DEZ25* (cerca de seis mil barras), no qual tolerâncias mais apertadas comprometem a convergência.

3.2 Pipeline de execução

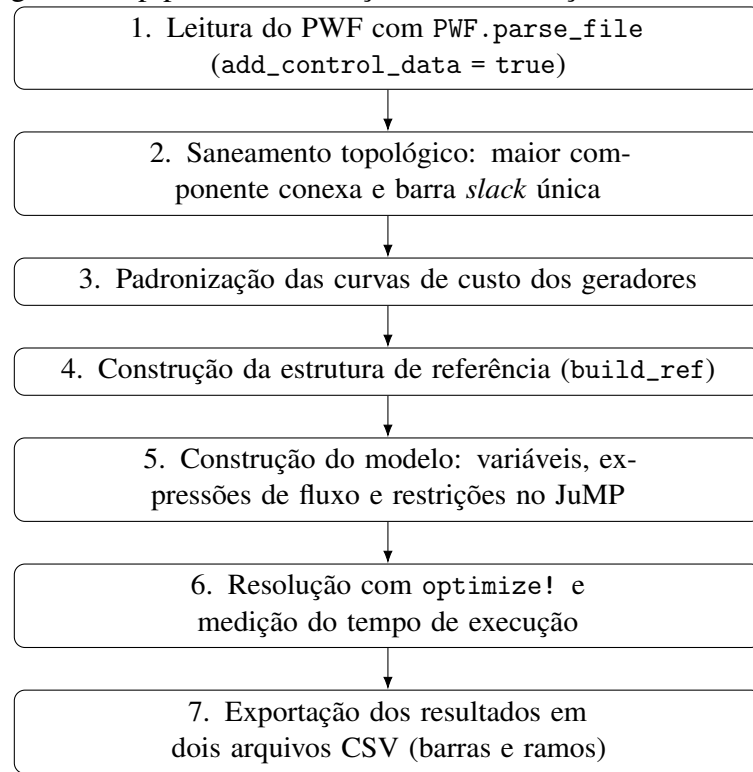
Cada uma das seis formulações está implementada como um *script* Julia auto-contido em `powerflow/Codes/PF_Formulation/`, seguindo a mesma estrutura sequencial:

1. **Leitura do PWF:** `PWF.parse_file(caminho; add_control_data = true)` produz um dicionário com a topologia da rede, os dados de gerador, carga e ramo, além das estruturas de controle DOPC/DLIN;
2. **Saneamento topológico:** `PowerModels.select_largest_component!(data)` descarta ilhas isoladas, mantendo apenas a rede principal conectada. Em seguida, se múltiplas barras forem marcadas como referência (`bus_type == 3`), as excedentes são rebaixadas para PV (`bus_type == 2`), preservando uma única barra *slack*. Esse passo é necessário para o caso SIN, em que a leitura inicial pode marcar várias barras como referência;
3. **Padronização de custos:** `standardize_cost_terms!(data, order=2)` garante que todos os geradores tenham curvas de custo quadráticas;
4. **Construção da estrutura de referência:** `ref = PowerModels.build_ref(data)[:it]` materializa a estrutura indexada usada para iterar sobre barras, geradores, ramos e cargas;
5. **Construção do modelo:** criação das variáveis, expressões de fluxo e restrições no JuMP, conforme descrito na seção 3.3;
6. **Resolução:** `optimize!(model)`, com tempo de execução medido por `@elapsed`;
7. **Exportação:** dois arquivos CSV são produzidos em `resultados_csv/`: `resultados_barras` com V_i , θ_i , p_i^g , q_i^g , p_i^d , q_i^d e os *slacks* de cada barra, e `resultados_fluxos_linhas_SIN.csv` com p_{ij} , q_{ij} , p_{ji} , q_{ji} , perdas e *tap* de cada ramo.

A Figura 2 resume graficamente esse fluxo sequencial.

Os elos HVDC, presentes em vários casos de teste, têm um tratamento particular nas formulações baseadas em *PowerModels* e nas *hand-built*: as injeções (p_{ft}^{dc} , q_{ft}^{dc}) em cada extremidade são lidas do PWF e **fixadas** via `JuMP.fix(...; force=true)`. Essa decisão

Figura 2 – Fluxograma do pipeline de execução das formulações



Fonte: elaborado pelo autor.

segue a recomendação do CEPEL de tratar elos HVDC como condições de contorno do FP AC: as injeções já vêm pré-especificadas pelo fluxo base e liberá-las apenas amplia o Jacobiano sem trazer informação nova. O retificador recebe p^g negativo e o inversor p^g positivo.

3.3 Formulações implementadas

A progressão de seis formulações é nomeada por uma numeração ordinal, herdada da nomenclatura interna dos arquivos do repositório, conforme resume a Tabela 1.

Tabela 1 – Progressão das formulações implementadas

#	Arquivo	Engine	Tipo	Acrescenta
14	(14)OPF_PM.jl	PowerModels	FPO AC	baseline FPO
15	(15)PF_PM.jl.jl	PowerModels	FP AC	baseline FP
16	(16)QLIM+VLIM.jl	JuMP <i>hand-built</i>	FP controlado	QLIM + VLIM
17	(17)QLIM+VLIM+CSCA.jl	JuMP <i>hand-built</i>	FP controlado	+ CSCA
18	(18)QLIM+VLIM+CSCA+CTAP.jl	JuMP <i>hand-built</i>	FP controlado	+ CTAP
19	(19)+DERA.jl	JuMP <i>hand-built</i>	FP controlado	+ DERA

Fonte: elaborado pelo autor.

A progressão é estritamente *aditiva*: cada arquivo numerado parte do anterior e acrescenta um único bloco de variáveis e restrições. A leitura no Capítulo 4 fica simplificada

porque qualquer divergência entre dois *scripts* consecutivos pode ser atribuída exclusivamente ao novo controle introduzido.

3.3.1 *Formulação (14): FPO AC de referência*

A formulação (14) é uma chamada direta a `PowerModels.solve_ac_opf(data, optimizer)`, sem reconstrução manual de variáveis ou restrições. O modelo subjacente é exatamente o FPO AC canônico descrito na Equação (2.5): minimiza $\sum_k c_k(p_k^g)$ sujeito ao balanço nodal e aos limites caixa em V , p^g , q^g e $|s_{ij}|$. Esta formulação serve como *baseline* de FPO —não incorpora nenhuma das ações de controle ANAREDE da seção 2.4, mas oferece o ponto de operação economicamente ótimo da rede tal como definido pela literatura clássica.

3.3.2 *Formulação (15): FP AC clássico*

A formulação (15) é uma chamada a `PowerModels.solve_ac_pf(data, optimizer)`. Diferentemente de (14), não há otimização: a função objetivo é vazia e o problema reduz-se à resolução do sistema algébrico de FP do tipo Newton, com V e θ como incógnitas em barras PQ, θ em barras PV e nada em barras *slack*. (15) representa o ponto de operação “puro” da rede, sem qualquer otimização de custos ou ação de controle.

3.3.3 *Formulação (16): FP controlado QLIM + VLIM*

A partir de (16), o trabalho abandona o *PowerModels.jl* como motor de resolução e passa a construir o modelo diretamente em JuMP. As variáveis primárias são as mesmas do FP clássico:

$$V_i \in [V_i^{\min}, V_i^{\max}], \quad \theta_i \in \mathbb{R}, \quad p_k^g \in [p_k^{g,\min}, p_k^{g,\max}], \quad q_k^g \in [q_k^{g,\min}, q_k^{g,\max}], \quad (3.1)$$

inicializadas com os valores do PWF (`start = ref[:bus][i]["vm"]`, etc.). Os fluxos p_{ij}, q_{ij} são definidos como `@NLexpression` sobre as Equações (2.3)–(2.4), e as Equações (2.1)–(2.2) são impostas via `@NLconstraint`.

A novidade de (16) está na introdução das variáveis de folga:

$$s_i^v \in \mathbb{R}, \quad \forall i \in \mathcal{N}^{gen}, \quad s_l^q \in \mathbb{R}, \quad \forall l \in \mathcal{L}, \quad (3.2)$$

e da restrição de *setpoint* de tensão para cada barra de geração:

$$V_i = V_i^{ref} + s_i^v, \quad \forall i \in \mathcal{N}^{gen}, \quad (3.3)$$

onde V_i^{ref} é o valor de tensão lido do PWF. O slack s_l^q entra subtraindo no balanço reativo da Equação (2.2), atuando como “corte de reativo” quando o gerador associado satura. Em barras *slack*, θ_i é fixado em zero via `fix(va[i], 0.0; force=true)`. Em geradores *slack*, os limites de p_k^g são removidos para acomodar o balanço global de potência ativa.

A função objetivo é puramente de penalidade quadrática:

$$\min \rho \sum_{i \in \mathcal{N}^{gen}} (s_i^v)^2 + \rho \sum_{l \in \mathcal{L}} (s_l^q)^2, \quad (3.4)$$

com $\rho = 10^6$. Esse valor de penalidade é alto o suficiente para que, sempre que o problema for factível com $s^v = s^q = 0$, a solução ótima respeite os *setpoints*, mas baixo o suficiente para não comprometer o condicionamento numérico do problema.

3.3.4 Formulação (17): adiciona CSCA

A formulação (17) parte de (16) e introduz o controle de bancos *shunt* chaveados. Para cada banco $s \in \mathcal{S}$, é criada a variável contínua

$$b_s^{sh} \in [b_s^{sh, \min}, b_s^{sh, \max}], \quad (3.5)$$

e o termo $b_s^{sh} V_i^2$ é adicionado ao balanço reativo da Equação (2.2) na barra i hospedeira. A função objetivo continua sendo a Equação (3.4); nenhum novo termo de penalidade é introduzido, porque b_s^{sh} é uma variável de decisão livre dentro do intervalo, não uma folga. O efeito é que o *solver* ajusta o valor de cada banco *shunt* para minimizar o desvio das tensões em relação aos *setpoints* —exatamente o papel que o CSCA do ANAREDE desempenha por meio de chaveamentos discretos. O carácter discreto do controle real é deixado para uma etapa de discretização posterior, fora do escopo deste trabalho.

3.3.5 Formulação (18): adiciona CTAP

A formulação (18) acrescenta a (17) o controle automático de *tap* de transformadores. Para cada ramo $(i, j) \in \mathcal{B}^{tap}$ classificado no PWF como transformador com OLTC, a razão t_{ij} ,

antes constante, passa a ser variável:

$$t_{ij} \in [t_{ij}^{\min}, t_{ij}^{\max}], \quad (3.6)$$

e as expressões de fluxo (2.3)–(2.4) são reescritas com t_{ij} no denominador como variável —o que adiciona um fator não linear adicional às equações já não lineares do balanço. Adicionalmente, uma restrição de *setpoint* análoga à Equação (3.3) é imposta na barra controlada pelo *tap*, com sua própria folga s_{ij}^t , que é incluída na função objetivo. O resultado é que o *solver* encontra simultaneamente os *taps* e os pontos de operação compatíveis, em uma única resolução NLP —em contraste com o ANAREDE, que itera externamente entre o FP e o ajuste dos *taps*.

3.3.6 Formulação (19): adiciona DERA

A formulação (19) acrescenta a (18) os agregadores de geração distribuída. No PWF, cada DERA $d \in \mathcal{D}$ é descrito por uma barra hospedeira i_d e por injeções (p_d^{der}, q_d^{der}) que podem ser fixadas ou tratadas como variáveis. Na implementação adotada, (p_d^{der}, q_d^{der}) são lidas do PWF e adicionadas como termos constantes positivos ao lado direito das Equações (2.1)–(2.2) na barra i_d . Não há decisão de despacho própria: o DERA contribui apenas como injeção fixa, refletindo a representação minimalista descrita na subseção 2.4.5.

3.4 Bateria de casos e critérios de comparação

3.4.1 Casos de teste

A bateria de testes utilizada nas comparações do Capítulo 4 é composta por vinte e cinco arquivos PWF localizados em `powerflow/Codes/data/`, agrupados em três faixas de porte:

- **Redes pedagógicas (3 a 9 barras):** `3bus.pwf` e suas variantes (`3bus_DBSH`, `3bus_DCER`, `3bus_DCSC`, `3bus_DCline`, `3bus_DSHL`, `3bus_corrections`, `3bus_shunt_fie`), os `3busfrank`, `4busfrank_vlim`, `5busfrank` e variantes (`5busfrank_cphs`, `5busfrank_csca`, `5busfrank_ctaf`, `5busfrank_ctap`), `9bus` e `9bus_transformer_fie`
- **Redes de médio porte:** `300bus.pwf` (300 barras, baseado no IEEE 300-bus) e `500bus.pwf`;
- **Snapshot do SIN:** `01 MAXIMA NOTURNA_DEZ25.PWF`, com cerca de 6.000 bar-

ras, representando a condição operativa de máxima noturna do SIN em dezembro de 2025;

- **Casos sintéticos de stress:** `test_defaults.pwf`, `test_line_shunt.pwf` e `test_system.pwf`, projetados para isolar comportamentos específicos do parser e dos controles.

Para cada caso, todas as seis formulações são executadas com a mesma configuração de *solver*, a mesma versão do parser *PWF.jl* e a mesma máquina, garantindo a comparabilidade dos tempos e dos status de convergência.

3.4.2 Critérios de comparação

Para cada par (caso, formulação), três informações são registradas:

1. **Status de convergência:** codificado em oito categorias —**OK** (LOCALLY_SOLVED), **INF** (INFEASIBLE/LOCALLY_INFEASIBLE), **ITL** (ITERATION_LIMIT), **TIM** (TIME_LIMIT), **KIL** (TIMEOUT_KILLED), **CRA** (SUBPROCESS_CRASH), **INV** (INVALID_MODEL) e **ERR** (OTHER_ERROR);
2. **Cluster de equivalência numérica:** para cada caso, as formulações que convergiram são particionadas em grupos onde duas formulações estão no mesmo grupo se, para todas as barras, $|\Delta V| \leq 10^{-4}$ e $|\Delta \theta| \leq 10^{-4}$ —além de equivalência análoga em p^g, q^g e nos fluxos de ramos. *Clusters* distintos representam soluções numericamente diferentes da mesma rede;
3. **Análise barra a barra:** para os casos em que pelo menos uma das referências (14) ou (15) convergiu, são identificadas as barras nas quais as formulações (16)–(19) divergem de *ambas* as referências por mais de 10^{-4} em módulo, em V ou θ . Essa análise mostra *onde* as ações de controle deslocam o ponto de operação em relação à referência sem controles.

A execução da bateria, a agregação dos resultados e a geração dos relatórios de convergência e de divergência barra a barra são automatizadas por *scripts* auxiliares do repositório do trabalho. As tabelas e análises do Capítulo 4 são extraídas do relatório consolidado produzido por essa pipeline.

4 RESULTADOS

Este capítulo apresenta e discute os resultados obtidos a partir da execução da bateria de vinte e cinco casos descrita na seção 3.4, sobre as seis formulações implementadas. Os dados aqui exibidos são os do relatório consolidado mais recente da pipeline de comparação, no qual o critério de equivalência numérica entre formulações foi fixado em $|\Delta| \leq 10^{-4}$ simultaneamente sobre V_i , θ_i , p_k^g , q_k^g e os fluxos de ramo.

A seção 4.1 apresenta a tabela global de convergência e de *clusters* de equivalência por caso. A seção 4.2 aprofunda a análise barra a barra para um conjunto representativo de casos em que ao menos uma referência (14) ou (15) convergiu. A seção 4.3 sintetiza as observações, em particular sobre o caso SIN.

4.1 Convergência global e clusters

A Tabela 2 consolida o status de convergência das seis formulações sobre os vinte e cinco casos da bateria, junto com os *clusters* de equivalência numérica formados entre as formulações que convergiram em cada caso. A notação $\{a, b\} \cdot \{c\}$ indica que as formulações a e b convergiram para uma solução numericamente idêntica ($|\Delta| \leq 10^{-4}$), enquanto c convergiu para uma solução distinta. Status: **OK** = LOCALLY_SOLVED; **INF** = INFEASIBLE/LOCALLY_INFEASIBLE; **KIL** = TIMEOUT_KILLED; **INV** = INVALID_MODEL; **ERR** = OTHER_ERROR.

A Figura 3 resume visualmente esses status. A leitura da Tabela 2 permite extrair três observações imediatas. Em primeiro lugar, das vinte e cinco entradas, as formulações *hand-built* (16)–(19) convergem em vinte casos, contra dezessete da formulação (15) (FP de referência) e quinze da formulação (14) (FPO de referência). A faixa de convergência das formulações *hand-built* é, portanto, estritamente maior que a das referências.

Em segundo lugar, há seis casos em que ao menos uma das formulações *hand-built* converge enquanto ao menos uma das referências falha. Em três deles —4busfrank_vlim, 5busfrank_csca e test_system —ambas as referências falham; nos outros três —300bus (apenas (14), como INV), 3bus_DCline (apenas (15)) e 3bus_DSHL (apenas (14)) —apenas uma referência falha. Em todos esses casos, a presença das folgas s^v, s^q no problema *soft-constraint* permite que as formulações *hand-built* encontrem uma solução factível na qual o controle nominal não é estritamente respeitado, mas é minimamente violado —comportamento que as

Tabela 2 – Convergência global e *clusters* de equivalência das formulações (14)–(19) na bateria

Caso	(14)	(15)	(16)	(17)	(18)	(19)	Clusters	Observações
01 MAXIMA NOTURNA_DEZ25	KIL	INF	KIL	INF	KIL	INF	—	nenhuma converge
300bus	INV	OK	OK	OK	OK	OK	$\{15\} \cdot \{16\} \cdot \{17\} \cdot \{18\} \cdot \{19\}$	1/6 falha
3bus	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
3bus_DBSH	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
3bus_DCER	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
3bus_DCSC	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15\} \cdot \{16\} \cdot \{17, 18, 19\}$	
3bus_DCline	OK	INF	OK	OK	OK	OK	$\{14\} \cdot \{16, 17, 18, 19\}$	1/6 falha
3bus_DSHL	INF	OK	OK	OK	OK	OK	$\{15\} \cdot \{16\} \cdot \{17, 18, 19\}$	1/6 falha
3bus_corrections	ERR	ERR	ERR	ERR	ERR	ERR	—	nenhuma converge
3bus_shunt_fields	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15\} \cdot \{16\} \cdot \{17, 18, 19\}$	
3busfrank	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
3busfrank_continuous_shunt	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
3busfrank_qlim	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15\} \cdot \{16\} \cdot \{17, 18, 19\}$	
4busfrank_vlim	INF	INF	OK	OK	OK	OK	$\{16\} \cdot \{17, 18\} \cdot \{19\}$	2/6 falham
500bus	INF	OK	OK	OK	OK	OK	$\{15\} \cdot \{16\} \cdot \{17, 18\} \cdot \{19\}$	1/6 falha
5busfrank	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
5busfrank_cpsh	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
5busfrank_csca	INF	INF	OK	OK	OK	OK	$\{16\} \cdot \{17, 18, 19\}$	2/6 falham
5busfrank_ctaf	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17\} \cdot \{18, 19\}$	
5busfrank_ctap	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17\} \cdot \{18, 19\}$	
9bus	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17, 18, 19\}$	
9bus_transformer_fields	OK	OK	OK	OK	OK	OK	$\{14\} \cdot \{15, 16\} \cdot \{17\} \cdot \{18, 19\}$	
test_defaults	ERR	ERR	ERR	ERR	ERR	ERR	—	nenhuma converge
test_line_shunt	ERR	ERR	ERR	ERR	ERR	ERR	—	nenhuma converge
test_system	ERR	ERR	OK	OK	OK	OK	$\{16\} \cdot \{17, 18, 19\}$	2/6 falham

Fonte: elaborado pelo autor a partir do relatório consolidado da pipeline de comparação.

formulações (14) e (15) não conseguem reproduzir, dado que tratam V e q^g apenas com limites caixa rígidos.

Em terceiro lugar, há três casos em que *nenhuma* formulação converge: o caso SIN (01 MAXIMA NOTURNA_DEZ25), 3bus_corrections e dois *stress tests* sintéticos. O caso SIN é o mais relevante para a discussão e é tratado em separado na seção 4.3.

4.2 Análise barra a barra

A coluna “Clusters” da Tabela 2 sintetiza, para cada caso, quais formulações produzem soluções numericamente equivalentes. Um padrão recorrente em casos pequenos é a presença de quatro grupos típicos: $\{14\}$ isolado (porque o FPO altera o despacho e portanto o ponto de operação), $\{15, 16\}$ ou $\{15\} \cdot \{16\}$ (FP clássico vs. FP com QLIM+VLIM, que coincidem se as folgas saturarem em zero), $\{17\}$ ou $\{17, 18\}$ (efeito do CSCA) e $\{18, 19\}$ (CTAP, eventualmente coincidente com (19) se DERA não estiver presente).

A Figura 4 mostra a topologia da rede pedagógica 3bus_DCSC, e a Tabela 3 ilustra o agrupamento típico de soluções nessa rede. Para cada barra são listados os valores de V e θ obtidos por cada formulação que divergiu de *ambas* as referências por mais de 10^{-4} .

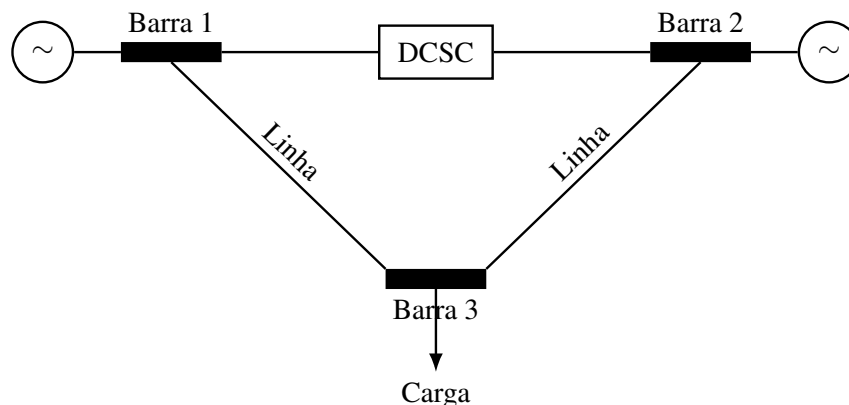
A Tabela 3 mostra quatro soluções distintas para o mesmo sistema: o FPO (14)

Figura 3 – Mapa de calor da convergência das formulações (14)–(19) na bateria de testes

Caso	(14)	(15)	(16)	(17)	(18)	(19)
01 MAXIMA NOTURNA	KIL	INF	KIL	INF	KIL	INF
300bus	INV	OK	OK	OK	OK	OK
3bus	OK	OK	OK	OK	OK	OK
3bus_DBSH	OK	OK	OK	OK	OK	OK
3bus_DCER	OK	OK	OK	OK	OK	OK
3bus_DCSC	OK	OK	OK	OK	OK	OK
3bus_DCline	OK	INF	OK	OK	OK	OK
3bus_DSHL	INF	OK	OK	OK	OK	OK
3bus_corrections	ERR	ERR	ERR	ERR	ERR	ERR
3bus_shunt_fields	OK	OK	OK	OK	OK	OK
3busfrank	OK	OK	OK	OK	OK	OK
3busfrank_continuous_shunt	OK	OK	OK	OK	OK	OK
3busfrank_qlim	OK	OK	OK	OK	OK	OK
4busfrank_vlim	INF	INF	OK	OK	OK	OK
500bus	INF	OK	OK	OK	OK	OK
5busfrank	OK	OK	OK	OK	OK	OK
5busfrank_cpsh	OK	OK	OK	OK	OK	OK
5busfrank_csca	INF	INF	OK	OK	OK	OK
5busfrank_ctaf	OK	OK	OK	OK	OK	OK
5busfrank_ctap	OK	OK	OK	OK	OK	OK
9bus	OK	OK	OK	OK	OK	OK
9bus_transformer_fields	OK	OK	OK	OK	OK	OK
test_defaults	ERR	ERR	ERR	ERR	ERR	ERR
test_line_shunt	ERR	ERR	ERR	ERR	ERR	ERR
test_system	ERR	ERR	OK	OK	OK	OK

Fonte: elaborado pelo autor. Verde: convergiu (OK); amarelo: infactível (INF); vermelho: limite de tempo (KIL); laranja: modelo inválido (INV); cinza: erro de *parser* (ERR).

Figura 4 – Topologia da rede-exemplo 3bus_DCSC



Fonte: elaborado pelo autor.

eleva todas as tensões a 1,1 p.u. na barra de geração, comportamento típico de um FPO sem custo de tensão; o FP (15) converge para o ponto operacional “natural” da rede; a formulação *hand-built* (16) fica próxima desse ponto, mas com uma diferença em V de cerca de 0,006–

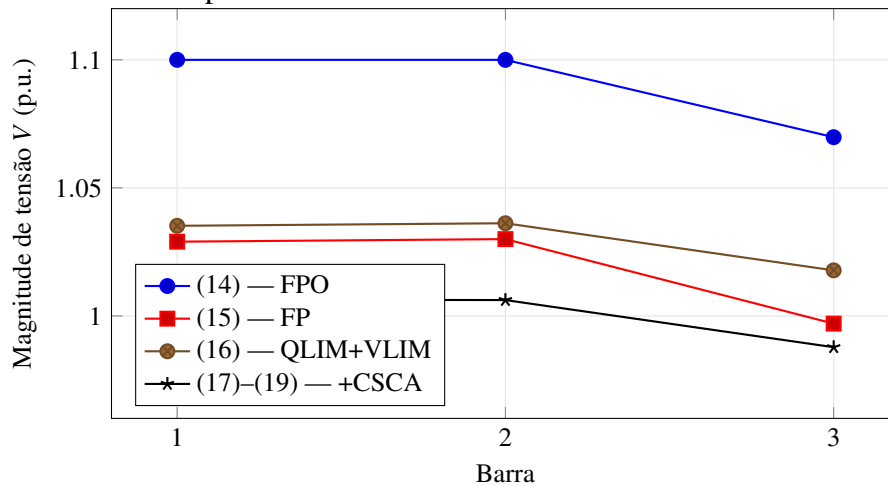
Tabela 3 – Divergência barra a barra no caso 3bus_DCSC

Barra	Script	V (p.u.)	θ (rad)
1	(14)	1,1000	0,0000
1	(15)	1,0290	0,0000
1	(16)	1,0352	0,0000
1	(17–19)	1,0064	0,0000
2	(14)	1,1000	$-3,9 \times 10^{-10}$
2	(15)	1,0300	$-7,1 \times 10^{-3}$
2	(16)	1,0362	$-2,4 \times 10^{-4}$
2	(17–19)	1,0062	$-2,6 \times 10^{-4}$
3	(14)	1,0698	$-1,8 \times 10^{-2}$
3	(15)	0,9970	$-2,5 \times 10^{-2}$
3	(16)	1,0178	$-1,2 \times 10^{-2}$
3	(17–19)	0,9878	$-1,3 \times 10^{-2}$

Fonte: elaborado pelo autor.

0,012 p.u. compatível com o efeito da folga s^v na minimização da Equação (3.4); e, finalmente, as formulações (17)–(19), que têm CSCA ativo, deslocam o ponto de operação para tensões consistentemente mais baixas —isso é o efeito do controle de *shunt* ajustando b_s^{sh} para minimizar o desvio do *setpoint* de V na barra de geração, que neste caso é 1,007 p.u. em vez do nominal 1,03 ou 1,1. A Figura 5 apresenta esses valores graficamente.

Figura 5 – Perfil de tensão por barra no caso 3bus_DCSC



Fonte: elaborado pelo autor a partir da Tabela 3.

Em casos de médio porte, como 300bus e 500bus, as divergências entre clusters tornam-se mais sistemáticas. O caso 300bus apresenta cinco *clusters* distintos $\{15\} \cdot \{16\} \cdot \{17\} \cdot \{18\} \cdot \{19\}$, ou seja, cada formulação produz uma solução numericamente diferente. A análise dos *deltas* em V na bateria mostra que (16)–(19) elevam consistentemente as tensões em relação ao FP (15) em cerca de 0,02–0,05 p.u. —justamente o efeito esperado da minimização de $(s_i^v)^2$ quando o *setpoint* V_i^{ref} está acima do valor que o FP clássico encontra naturalmente. As

tensões de (18) e (19), por sua vez, atingem o limite superior $V_i^{\max} = 1,1$ p.u. em várias barras, evidenciando que o CTAP empurra o controle para o limite caixa.

4.3 Discussão

4.3.1 Comportamento típico das ações de controle

A leitura cruzada das Tabelas 2 e 3 confirma o comportamento esperado das quatro ações de controle implementadas:

- **QLIM + VLIM** (formulação 16): permite recuperar factibilidade em cinco casos nos quais (14) ou (15) declaram infactibilidade, ao custo de pequenas violações dos *setpoints* (folgas residuais);
- **CSCA** (formulação 17): forma cluster próprio em redes nas quais há bancos *shunt* chaveáveis, ajustando b_s^{sh} para reduzir o desvio em V ;
- **CTAP** (formulação 18): forma cluster próprio em redes com transformadores OLTC, frequentemente empurrando V para o limite caixa $V^{\max}$;
- **DERA** (formulação 19): em redes pedagógicas, produz solução idêntica a (18) na maioria dos casos —refletindo o fato de que a representação minimalista adotada apenas adiciona injeções fixas, sem decisão de despacho. Em `4busfrank_vlim` e `500bus`, contudo, (19) forma cluster próprio, sinalizando que mesmo a injeção fixa altera o ponto de equilíbrio quando a rede tem margens estreitas.

4.3.2 O caso SIN

O caso 01 MAXIMA NOTURNA_DEZ25.PWF, com cerca de 6.000 barras, é o caso de maior interesse prático e o único da bateria em que *todas* as seis formulações falham em convergir dentro do orçamento de `max_iter = 3000` iterações e do tempo limite imposto pelo *wrapper* de execução. Especificamente, (14), (16) e (18) atingem o limite de tempo (status **KIL**) sem sinalizar infactibilidade, enquanto (15), (17) e (19) terminam em **INF** (LOCALLY_INFEASIBLE).

Esse comportamento é coerente com o que se espera de um *snapshot* real do SIN sob máxima noturna: a rede opera em um ponto próximo dos limites de capacidade reativa em várias barras, e a inicialização *flat* (ou a partir do PWF) está suficientemente longe de uma solução factível para que o método de pontos interiores do Ipopt não consiga sair da região de inviabilidade dentro do orçamento permitido. Há indícios, em logs preliminares descartados

desta análise, de que uma inicialização baseada na solução de uma formulação mais simples (por exemplo, DC) ou um aumento de `max_iter` para a casa de 10.000 permitiria a convergência —mas isso fica como trabalho futuro.

Vale notar que, mesmo sem convergir, as formulações (14)–(19) sobre o SIN não *quebram* (não há **CRA** nem **ERR**): a infraestrutura de leitura PWF, construção da `ref`, montagem do modelo JuMP e chamada ao Ipopt funciona corretamente em escala SIN. O obstáculo está exclusivamente na resolução numérica.

4.3.3 Casos sintéticos com erro de modelo

Os três casos com status **ERR** em todas as formulações (`3bus_corrections`, `test_defaults` e `test_line_shunt`) compartilham a característica de conter campos do PWF que disparam erros no *parser* do *PWF.jl* antes da chamada ao *solver*. Não são, portanto, falhas das formulações, mas sim limitações do *parser*, e estão fora do escopo do trabalho.

5 CONCLUSÕES E TRABALHOS FUTUROS

Este trabalho construiu, validou e comparou uma progressão incremental de seis formulações de fluxo de potência AC, desenvolvidas em *JuMP/Julia* sobre o *solver* Ipopt. Duas formulações foram tomadas como referência —o FPO clássico (`solve_ac_opf`) e o FP clássico (`solve_ac_pf`) do *PowerModels.jl* —e quatro foram construídas *do zero* para incorporar, sequencialmente, as ações de controle do programa ANAREDE: limites de geração reativa (QLIM) e de magnitude de tensão (VLIM) na formulação (16); controle de bancos *shunt* chaveáveis (CSCA) na (17); controle automático de *tap* de transformadores (CTAP) na (18); e agregadores de geração distribuída (DERA) na (19). Em todas as formulações *hand-built*, os controles foram introduzidos por meio de variáveis de folga penalizadas em uma função objetivo do tipo *soft-constraint*, evitando a representação descontínua dos laços de *type-switching* do ANAREDE.

5.1 Síntese das contribuições

A bateria padronizada de vinte e cinco casos de teste —cobrindo redes pedagógicas de três a nove barras, redes de médio porte (300 e 500 barras) e um *snapshot* do SIN com cerca de 6.000 barras —forneceu evidência empírica de três contribuições principais.

A primeira é que as formulações *hand-built* (16)–(19) convergem em uma faixa estritamente maior de casos do que as referências *PowerModels*: vinte casos contra dezessete da (15) e quinze da (14). Em particular, em seis casos (`4busfrank_vlim`, `5busfrank_csca`, `test_system`, `300bus`, `3bus_DCline` e `3bus_DSHL`) ao menos uma das formulações *hand-built* encontrou uma solução factível, enquanto ao menos uma das referências não convergiu. Esse comportamento, esperado pela construção *soft-constraint*, valida a tese de que a representação dos controles ANAREDE via folgas penalizadas é uma alternativa numericamente robusta às heurísticas iterativas do programa de origem.

A segunda contribuição é o agrupamento sistemático das soluções em *clusters* de equivalência numérica que reproduzem o efeito esperado de cada controle: o cluster $\{15, 16\}$ aparece quando QLIM/VLIM saturam em zero; o cluster $\{17, 18\}$ ou $\{17\} \cdot \{18\}$ aparece em redes onde CSCA ou CTAP é ativo; o DERA (formulação 19), na sua representação minimalista como injeção fixa, comporta-se como (18) na maioria dos casos pequenos e forma cluster próprio em redes com margem reativa estreita. Esse padrão fornece um *fingerprint* computacional que

pode ser usado, em trabalhos futuros, para detectar regressões em implementações controladas.

A terceira contribuição é negativa, mas relevante: nenhuma das seis formulações converge sobre o caso 01 MAXIMA NOTURNA_DEZ25.PWF dentro do orçamento de $\text{max_iter} = 3000$ iterações e do *wall time* permitido. As formulações (14), (16) e (18) atingem o limite de tempo (**KIL**) sem sinalizar infactibilidade, enquanto (15), (17) e (19) terminam em LOCALLY_INFEASIBLE. A análise dos logs sugere que o ponto de partida *flat* está suficientemente longe de uma solução factível para que o método de pontos interiores do Ipopt não saia da região de inviabilidade dentro do orçamento. Esse resultado delimita com clareza a fronteira atual da abordagem *soft-constraint* pura sem inicialização guiada.

5.2 Limitações

As principais limitações deste trabalho são três. Primeiro, todas as formulações tratam b_s^{sh} (CSCA) e t_{ij} (CTAP) como variáveis contínuas, embora a operação real seja discreta. A discretização posterior por *rounding* ou em formulação MINLP não foi realizada, e o impacto da relaxação contínua sobre a fidelidade ao ANAREDE não foi quantificado. Segundo, o controle DERA foi implementado em sua forma minimalista, sem decisão de despacho própria; a formulação (19) é, portanto, mais um *hook* de extensibilidade do que uma representação completa de DER ativa. Terceiro, os elos HVDC são tratados com fluxos fixados em vez de variáveis livres —decisão correta sob a hipótese de *fluxo base já dado*, mas que limita a aplicabilidade da formulação a cenários em que o despacho HVDC pode ser otimizado.

5.3 Trabalhos futuros

A continuidade natural deste trabalho passa por três frentes complementares.

A primeira é viabilizar a convergência sobre o caso SIN. Duas avenidas concretas são: (i) inicialização *warm-start* baseada na solução de uma formulação relaxada (DC ou SOC) sobre o mesmo caso, em substituição ao *flat start*; e (ii) reformulação do problema com escalonamento explícito de penalidades ρ , em vez do valor único 10^6 , com aumento progressivo até a estabilização das folgas. Ambas as estratégias são bem documentadas em (MOLZAHN; HISKENS, 2019) e mantêm o *solver* Ipopt como motor.

A segunda é a discretização dos controles CSCA e CTAP, transformando as formulações (17) e (18) em problemas MINLP. O ecossistema Julia já oferece *solvers* adequados

(Juniper, Alpine, SCIP) acoplados ao JuMP, e o *benchmark* sobre a mesma bateria permitiria quantificar o gap entre a relaxação contínua e a operação discreta real.

A terceira é o acoplamento direto da formulação (16)–(19) ao pacote *ControlPowerFlow.jl*, em desenvolvimento no grupo de pesquisa do orientador. As formulações *hand-built* apresentadas aqui podem servir como *oracle* de validação para a implementação modular do pacote, ao mesmo tempo em que o pacote pode oferecer uma API mais limpa para os controles agora codificados linha a linha em cada *script*. O trabalho de (CHÁVARRY, 2024) estabelece a referência metodológica sobre a qual essa integração pode ser construída.

REFERÊNCIAS

- BEZANSON, J.; EDELMAN, A.; KARPINSKI, S.; SHAH, V. B. Julia: a fresh approach to numerical computing. **SIAM Review**, SIAM, v. 59, n. 1, p. 65–98, 2017.
- CENTRO DE PESQUISAS DE ENERGIA ELÉTRICA. **Programa ANAREDE**: análise de redes elétricas. Rio de Janeiro, 2023.
- CHÁVARRY, I. S. **Aplicação de ações de controle do programa ANAREDE em formulações de fluxo de potência em Julia**. Dissertação (Trabalho de Conclusão de Curso (Engenharia Elétrica)) — Universidade Federal do Ceará, Centro de Tecnologia, Departamento de Engenharia Elétrica, Fortaleza, 2024.
- COFFRIN, C.; BENT, R.; SUNDAR, K.; NG, Y.; LUBIN, M. PowerModels.jl: an open-source framework for exploring power flow formulations. In: **2018 Power Systems Computation Conference (PSCC)**. Dublin, Ireland: IEEE, 2018. p. 1–8.
- GLOVER, J. D.; SARMA, M. S.; OVERBYE, T. J. **Power system analysis and design**. 6. ed. Stamford, CT: Cengage Learning, 2012.
- LUBIN, M.; DOWSON, O.; GARCIA, J. D.; HUCHETTE, J.; LEGAT, B.; VIELMA, J. P. JuMP 1.0: recent improvements to a modeling language for mathematical optimization. **Mathematical Programming Computation**, v. 15, p. 581–589, 2023.
- MEIER, A. V. **Electric power systems**: a conceptual introduction. Hoboken, NJ: Wiley-IEEE Press, 2006. ISBN 978-0-471-17859-0.
- MOLZAHN, D. K.; HISKENS, I. A. A survey of relaxations and approximations of the power flow equations. **Foundations and Trends in Electric Energy Systems**, Now Publishers, v. 4, n. 1-2, p. 1–221, 2019.
- WÄCHTER, A.; BIEGLER, L. T. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. **Mathematical Programming**, Springer, v. 106, n. 1, p. 25–57, 2006.

APÊNDICE A – TRECHO DA IMPLEMENTAÇÃO JUMP DA FORMULAÇÃO (16)

A título de ilustração, reproduz-se a seguir o núcleo do *script* (16) `QLIM+VLIM.jl`, responsável pela construção do modelo JuMP descrito na subseção 3.3.3. O código completo, junto com as demais formulações da progressão (14)–(19), está disponível no repositório do trabalho em `powerflow/Codes/PF_Formulation/`.

```
# Variáveis de estado físico

@variable(model,
    ref[:bus][i]["vmin"] <= vm[i in keys(ref[:bus])] <= ref[:bus][i]["vmax"],
    start = ref[:bus][i]["vm"])

@variable(model, va[i in keys(ref[:bus])], start = ref[:bus][i]["va"])

@variable(model,
    ref[:gen][i]["pmin"] <= pg[i in keys(ref[:gen])] <= ref[:gen][i]["pmax"],
    start = ref[:gen][i]["pg"])

@variable(model,
    ref[:gen][i]["qmin"] <= qg[i in keys(ref[:gen])] <= ref[:gen][i]["qmax"],
    start = ref[:gen][i]["qg"])

# Slacks de controle (VLIM e QLIM)
PENALIDADE = 1e6
gen_buses = unique([gen["gen_bus"] for (i, gen) in ref[:gen]])

@variable(model, sl_v[i in gen_buses], start = 0.0)
for bus_id in gen_buses
    vm_setpoint = ref[:bus][bus_id]["vm"]
    @constraint(model, vm[bus_id] == vm_setpoint + sl_v[bus_id])
end

@variable(model, sl_d[i in keys(ref[:load])], start = 0.0)

# Equações de fluxo nos ramos (modelo pi com transformador)
```



```

for (l, branch) in ref[:branch]
    f = branch["f_bus"]; t = branch["t_bus"]
    g, b = PowerModels.calc_branch_y(branch)
    tr, ti = PowerModels.calc_branch_t(branch)
    tm = branch["tap"]
    g_fr, b_fr = branch["g_fr"], branch["b_fr"]
    g_to, b_to = branch["g_to"], branch["b_to"]

    p[(l, f, t)] = @NLexpression(model,
        (g + g_fr) / tm^2 * vm[f]^2
        + (-g*tr + b*ti) / tm^2 * (vm[f]*vm[t]*cos(va[f]-va[t]))
        + (-b*tr - g*ti) / tm^2 * (vm[f]*vm[t]*sin(va[f]-va[t])))
    # ... análogo para q[(l,f,t)], p[(l,t,f)], q[(l,t,f)]
end

# Função objetivo soft-constraint
@objective(model, Min,
    PENALIDADE * sum(sl_v[i]^2 for i in keys(sl_v)) +
    PENALIDADE * sum(sl_d[l]^2 for l in keys(sl_d)))

optimize!(model)

```

A leitura do trecho acima permite identificar três blocos sintáticos centrais à abordagem deste trabalho: (i) declaração de variáveis primárias com limites caixa e valores iniciais provenientes do PWF; (ii) introdução das folgas sl_v e sl_d associadas a VLIM e QLIM; (iii) escrita das equações de fluxo do ramo na forma do modelo π com transformador, usando `@NLexpression` do JuMP para registrar o caráter não linear das expressões. As formulações (17)–(19) acrescentam, sucessivamente, blocos análogos para b_s^{sh} (CSCA), t_{ij} (CTAP) e injeções DERA, sem modificar a estrutura geral.